DSA：从 Lightning Indexer 到稀疏 Attention 训练

> **原始问题：**讲解清楚大模型训练里面的dsa技术，稀疏attention的一种

提问：2026/11/03 16:44　更新：2026/11/04 15:30　重要度：5/5

DSA：从 Lightning Indexer 到稀疏 Attention 训练

1. 先给结论与版本边界

这里的 DSA 指 **DeepSeek Sparse Attention**，以 DeepSeek-V3.2-Exp 原始技术报告和官方参考实现为依据，不把所有 Dynamic Sparse Attention 都视为同一算法，也不把 NSA 当成 DSA。

一句话：**每个 Query 先让一个低成本的 Lightning Indexer 为历史 Token 打分，选出 Top-k 个位置；昂贵的主 Attention 只在这些位置上计算，并在选中的集合内重新做 Softmax。Indexer 用注意力分布蒸馏来训练，主模型用语言建模 Loss 训练。**

关键不是"算完完整主注意力再把小值清零"，而是把"挑选位置"和"精算注意力"分开。主 Attention 交互量约为 O(Sk)，但原始 Indexer 仍有 $O(S^2)$ 扫描，不能声称整个 DSA 严格线性。

本课讲清算法、训练和系统代价；小例子用标准 Q/K/V 展示链式法则，不冒充完整 MLA 或 FP8 生产内核。原始报告的配方是从稠密检查点继续训练，不是已经证明任意模型从零稀疏训练都可直接套用。

2. 为什么要稀疏：昂贵的是什么

标准 Causal Attention 对第 t 个 Query 访问所有允许的 Key，下面采用从 0 开始的位置编号：

```
可见集合 C_t = {0,1,...,t}
z[t,s,h] = dot(Q[t,h], K[s,h]) / sqrt(d_qk)
A[t,:,h] = softmax(z[t,:,h])
O[t,h] = Σ_s A[t,s,h] V[s,h]
```

B 为 Batch、S 为序列长度、H_a 为主 Query 头数、d_qk/d_v 为主 QK/Value 维度。普通多头逻辑 shape 为 Q/K:[B,S,H_a,d_qk]、V/O:[B,S,H_a,d_v]。

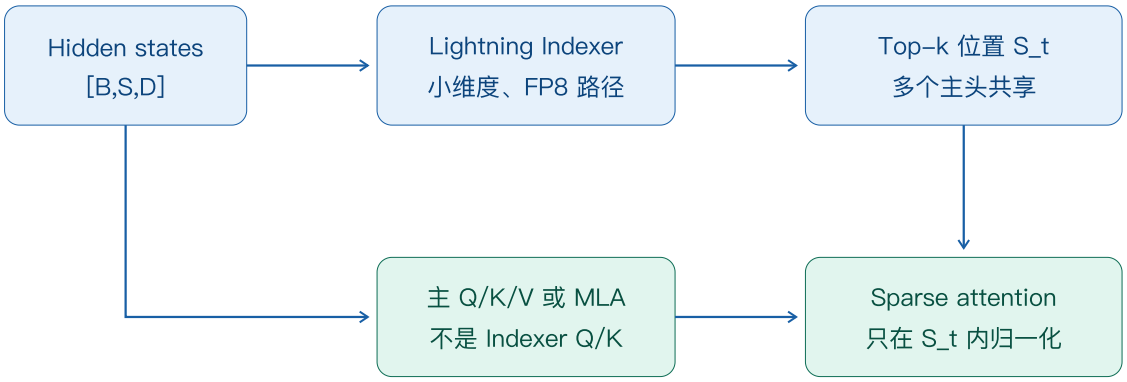
整段因果序列有 S(S+1)/2 个 Query-Key 对。FlashAttention 通过分块和在线 Softmax 避免保存整个注意力矩阵，但仍计算所有允许的对；DSA 则改变允许参与主注意力的集合，减少实际交互。

这两者解决不同问题：FlashAttention 主要优化 IO 和中间存储；DSA 同时引入选择算法和模型训练适配。可以用 Flash 风格内核执行稀疏注意力。

3. DSA 的两条计算路径

```
Hidden States
  ├─ Indexer: 小 Query/Key → 索引分数 I[t,s] → Top-k 位置 S_t
  └─ 主 Attention: 主 Q/K/V 或 MLA latent → 只访问 S_t → 输出 O
```

同一个 Query 的索引集合通常供该层多个主 Attention Heads 共享；各主头仍然计算自己的注意力权重。**共享选中的位置，不等于共享最终概率，也不等于 Indexer 只有一个头。**



训练 Indexer：KL 蒸馏；离散 Top-k 的索引不提供普通可导梯度

训练主模型：LM Loss → 稀疏 Attention → 被选 Q/K/V 的梯度

4. Lightning Indexer 公式逐项解释

原始报告的核心公式：

```
I[t,s] = Σ_{j=1..H_i} wI[t,j] × ReLU(dot(qI[t,j], kI[s]))
```

变量与逻辑 shape：

张量	Shape	职责
Hidden x	[B,S,D]	主模型输入表示
Indexer Query qI	[B,S,H_i,d_i]	每 Query 多个检索方向
Indexer Key kI	[B,S,d_i]	每历史 Token 一份共享索引表示
Query-dependent weights wI	[B,S,H_i]	合并多个索引头的分数
索引分数 I	[B,S,S]，逻辑 shape	每对 Query-Key 一个选择分数
索引列表 indices	[B,S,k]，尾部可 Padding	被选位置，不带主头维度

qI/kI 不是主 Attention 的 Q/K。报告用 Hidden 派生描述其来源；官方实现中 qI 复用 MLA 的低秩 Query 中间表示作为投影输入，kI 从 Hidden 投影并归一化，再做部分 RoPE 等处理。

一次索引打分包含三步：

1. 各索引头做 dot(qI,kI)，看候选 Key 是否与该检索方向相关。
2. 每个头的点积先经过 ReLU，再跨头加权求和。**ReLU 必须在跨头聚合之前**。
3. 得到一个标量 I[t,s]，用于候选位置的排序，而不是直接当主 Attention 概率。

wI 是学习出来的线性权重，不能默认它是 Softmax 概率，也不能默认所有权重非负。原始公式中的分数不需要加和为 1。Index head scaling、量化 scale 等需以实际实现为准，教学公式省略这些可吸收缩放，不省略 ReLU。

官方 V3.2-Exp 配置示例：H_i=64、d_i=128、k=2048；主 Query 头数为128，MLA KV latent rank为512。这些是一个模型配方，不是 DSA 定义要求的常数。“轻量”是相对昂贵的主 MLA 计算及低精度实现而言，不代表 Indexer FLOPs 为零。

5. Top-k：先满足语义，再挑重要位置

```
C_t = 同一个样本/segment 内、满足 causal 和自定义 mask 的可见位置
S_t = TopK(I[t,s], s∈C_t), 取 min(k, |C_t|) 个有效位置
```

未来 Token、Padding、跨 Packed Sequence 的位置必须在选择时屏蔽；主稀疏内核也要验证有效位置。不能先 Top-k 再只看“索引看起来在范围内”。

有一个很容易忽略的边界：早期 Query 的有效历史少于 k。若矩形 topk 固定返回 k 个索引，它可能把分数为负无穷的未来位置也作为占位补足数量。因此“Top-k 前 Mask”不意味着返回数组里永远没有无效占位，消费者必须使用有效长度或再次 Mask，不能给占位项分配概率。

相同分数的 tie-breaking 也需要考虑多 Rank 一致性。Top-k 可能保留自身，但“必须保留自身”“必须保留滑窗”不是原始 DSA 核心公式天然保证的规则，若要强制加入必须明确这是额外策略。

6. 选好以后：重新算主 Attention，不使用 Indexer 概率

对主头 h：

```
z[t,h,s] = dot(Q[t,h], K[s,h]) / sqrt(d_qk), s∈S_t
A[t,h,s] = exp(z[t,h,s]) / Σ_{r∈S_t} exp(z[t,h,r])
O[t,h]   = Σ_{s∈S_t} A[t,h,s] × V[s,h]
```

非选中位置在这次主 Attention 中为0。分母也只在 S_t 内计算。

所以稀疏结果一般不等于 Dense 结果：不只是把 Dense 概率中的小项清零，而是删掉一批项后重新归一化。即使保存大部分 Dense 概率质量，输出也有误差，尤其当被漏掉的 Value 很大或携带关键事实时。训练用于适应这种改变，不是证明两种 Attention 数学等价。

7. 前置补充：MQA、GQA、MLA 怎样与 DSA 拼在一起

一句话：MQA/GQA 决定每个历史 Token 存几组 K/V；MLA 决定怎样用共享的低秩 latent 表示历史并保留每头差异；DSA 决定当前 Query 读哪些历史 Token。不能把 DSA 报告所说“MLA 的 MQA 计算模式”误认成“整个模型变成标准 MQA”。

7.1 从 MHA 到 GQA 和 MQA：共享的是 K/V，不是 Q 和概率

令 `B` 为 batch、`S` 为缓存序列长度、`H_q` 为 Query 头数、`H_kv` 为 KV 头组数、`d` 为单头 K/V 宽度。设 `B=1、S=4096、H_q=8、d=128、BF16=2 Byte/元素`。先只讨论**单层推理 KV cache**，不包含训练保存的激活、梯度、分布式副本。

方案	Q 逻辑 shape	实际缓存的 K/V shape（各一份）	Q → KV 对应	每历史 Token 的 KV 元素
MHA	`[B,S,8,128]`	`[B,S,8,128]`	一头一组	`2×8×128=2048`
GQA-2	`[B,S,8,128]`	`[B,S,2,128]`	每 4 头共一组	`2×2×128=512`
MQA	`[B,S,8,128]`	`[B,S,1,128]`	8 头共一组	`2×128=256`

```
KV_cache_bytes = B × S × 2 × H_kv × d × bytes_per_element
```

量纲：`token × 元素/token × Byte/元素 = Byte`。有的代码把 K/V 逻辑广播为 `[B,S,H_q,d]`；如果仅为广播视图，不意味着缓存真的复制 `H_q` 份，朴素 materialize 则需额外计入临时峰值。

- **MQA（Multi-Query Attention）**：所有 Q 头共用一组完整的 K/V，但每头 `q_h` 不同，`score_h(s)=dot(q_h,k_s)/√d` 和 Softmax 一般仍不同。主要降低自回归解码时历史 KV 的缓存/读取带宽，可能牺牲表示能力。
- **GQA（Grouped-Query Attention）**：`H_q` 个 Q 头映射到 `H_kv` 个 KV 组。组号 `g(h)=floor(h/(H_q/H_kv))`，头 `h` 用 `k_{s,g(h)}`。`H_kv=H_q` 是 MHA；`H_kv=1` 是 MQA。GQA-2 示例的头映射是 `[0,0,0,0,1,1,1,1]`。组数是缓存带宽与质量的折中；原论文讨论从 MHA checkpoint 继续训练，不能把换组当成无损转换。
- **共同边界**：即便只有一组 KV，各 Q 头仍分别对可见历史打分/归一化；MQA/GQA **不会自动把主注意力的 S^2 有效 Query-Key 边减少为 Sk**，后者才是 DSA 选择位置带来的变化。



7.2 MLA：共享 latent，逐头重建内容 K/V

MLA（Multi-head Latent Attention）不是简单设 `H_kv=1`。把隐藏状态 `x_s:[D]` 压缩成共享内容潜变量 `c_s=W_DKV x_s:[d_c]`；另生成共享的 RoPE Key `kR_s:[d_r]`。各头有**自己的**恢复矩阵 `W_UK,h`、`W_UV,h`，以及内容 Query `qC_{t,h}` 和 RoPE Query `qR_{t,h}`：

```
kC_{s,h} = W_UK,h c_s                  [d_nope]
v_{s,h}  = W_UV,h c_s                  [d_v]
score_{t,h,s} = (qC_{t,h}·kC_{s,h} + qR_{t,h}·kR_s) × scale
O_{t,h} = Σ_s softmax_s(score_{t,h,s}) × v_{s,h}
```

`d_c` 为压缩宽度，`d_r` 为共享的位置 Key 宽度。Query 也可能低秩压缩，但不是 KV cache 减少的直接原因。位置编码单独保存，使**内容路径的矩阵吸收**成为可能：

```
qC_h · (W_UK,h c_s) = (W_UK,h^T qC_h) · c_s
Σ_s a_{h,s}(W_UV,h c_s) = W_UV,h (Σ_s a_{h,s}c_s)
```

所以在吸收式 decode 中，每头独立生成 `qAbs_h=W_UK,h^T qC_h:[d_c]`，与**同一份**缓存 `c_s` 点积，再加位置分数；按各头独立的概率加权 latent，最后乘各头 Value 恢复矩阵。即使共享 latent 与被选中的位置，也**没有共享逐头 Softmax 权重和输出**。有的实现还能把 Value 恢复合并进输出投影；具体是否融合取决于内核。DeepSeek-V3.2-Exp 官方可读推理代码把该 decode 分支注释为 `MQA decode`，但它不是标准 MQA 的"共享一套完整 K/V"；prefill 分支又可能显式展开各头 K/V，不能把两个分支的中间峰值混为一谈。

推理缓存口径（每层）：

```
MLA_cache_bytes = B × S × (d_c+d_r) × bytes_per_element
```

不再额外乘 `H_q`；但它不一定小于 MQA 的 `2d`，须代入具体维度比较。

7.3 同一组数字：缓存与一次稀疏读取

沿用 `B=1、S=4096、H_q=8、G=2、d=128、BF16`，另设 `d_c=512、d_r=64`（取自 DeepSeek MLA 的维度配置；`H_q=8` 只是教学假设，不是 DeepSeek 原模型的完整配置）。假设一次 DSA Query 选 `k=2048` 个位置，且读取可跨 Q 头复用：

单层方案	每历史 Token 字节	4096 Token 理论 KV cache	一次只读 2048 个位置的原始字节下界
MHA	`2×8×128×2=4096 B`	`16 MiB`	`8 MiB`
GQA-2	`2×2×128×2=1024 B`	`4 MiB`	`2 MiB`
MQA	`2×128×2=512 B`	`2 MiB`	`1 MiB`
MLA	`(512+64)×2=1152 B`	`4.5 MiB`	`2.25 MiB`

MLA 行代入：`4096 token × 576 元素/token × 2 Byte/元素 = 4,718,592 Byte = 4.5 MiB`。右列不是实测 HBM 流量：重读、缓存命中、Gather、对齐、量化 scale、Indexer Key、通信与工作区都未计入。此配置 MLA 缓存**比 MQA 大**并不矛盾：它保留了逐头内容恢复的自由度；不可笼统说 MLA 必定优于 MQA。该表只算**推理缓存**，不是训练显存峰值。

再用两历史 Token 和两头验证"共享缓存不等于共享分数"：忽略位置项并暂设 `scale=1`，令 `c_0=[1,0]、c_1=[0,1]`，两个头的 `qC` 都是 `[1,0]`，但 `W_UK,0=diag(2,1)`、`W_UK,1=diag(1,3)`。吸收后 `qAbs_0=[2,0]、qAbs_1=[1,0]`，因此：

```
头 0: scores=[2,0] → softmax≈[0.880797,0.119203]
头 1: scores=[1,0] → softmax≈[0.731059,0.268941]
```

相同的两份 latent、不同的头概率。若 DSA 选中集合是 `{0}`，两个头都只读 `c_0`；若是 `{0,1}`，保留上面两套不同概率。这个小例子演示矩阵吸收，不采用实际模型的 scale 或位置编码。

7.4 前反向与 DSA 的衔接

- MQA/GQA：组内多个 Q 头共享 K/V，Backward 中 `dK_group/dV_group` 要累加所有消费该组的头的贡献；若组跨 TP Rank，需要考虑复制和归约。
- MLA：`c_s`、`kR_s`、每头恢复矩阵仍需训练。推理只缓存 latent 不等于训练不保存展开 K/V 激活；是否物化及其峰值取决于 prefill、吸收式 kernel、重算策略。
- DSA：`Top-k indices:[B,S,k]` 不带主头维，选中位置及对应 latent 供多个主头复用；各头仍各算主分数/梯度。只读 `k` 个历史位置不等于只缓存 `k` 个 Token：后续 Query 可能另选旧位置，完整历史 latent 与额外 Indexer Key 一般仍需保留。

第 8 节手算采用普通 Q/K/V 稀疏注意力，第 12 节 FLOPs 也是假设普通多头的解析模型；都**不是** MLA-DSA 生产内核的精确 FLOPs。实现验收要分别记录 Indexer/Top-k、实际跳过的 Tile、KV/latent 读取、反向 dK/dV 归约、TP/CP 通信与训练峰值，而不是套用“推理 KV cache 缩减比例”。

```
# 最小形状/缓存检查：教学口径，非真实 DSA Kernel
B, S, Hq, G, d, dc, dr, itemsize = 1, 4096, 8, 2, 128, 512, 64, 2
assert Hq % G == 0
assert [h // (Hq // G) for h in range(Hq)] == [0,0,0,0,1,1,1,1]
cache_mib = {
    "MHA": B*S*2*Hq*d*itemsize / 2**20,
    "GQA": B*S*2*G*d*itemsize / 2**20,
    "MQA": B*S*2*d*itemsize / 2**20,
    "MLA": B*S*(dc+dr)*itemsize / 2**20,
}
print(cache_mib)  # MHA=16, GQA=4, MQA=2, MLA=4.5 MiB
```

资料与边界：MQA 原论文 https://arxiv.org/abs/1911.02150；GQA 原论文 https://arxiv.org/abs/2305.13245；DeepSeek-V2 MLA 原论文第 2.1 节 https://arxiv.org/html/2405.04434v5#S2.SS1；DSA 报告及可读参考实现见本章末尾。核对日期：2026-11-04。

8. 一个可手算的完整例子

只展示最后一个 Query：B=1，S=4，t=3，允许看位置0、1、2、3；k=2。Indexer 有2个头，每头维度2。以下向量视为已完成各自投影/位置处理后的值，省略量化。

8.1 Indexer 如何选出两个位置

```
qI[3,0] = [1,0]
qI[3,1] = [0,1]
wI[3,:] = [1,0.5]
```

Key位置 s	kI[s]	头0 dot 后 ReLU	头1 dot 后 ReLU	I[3,s]
0	[2,0]	2	0	1×2+0.5×0=2
1	[0,2]	0	2	1×0+0.5×2=1
2	[−1,−1]	0	0	0
3	[1,1]	1	1	1×1+0.5×1=1.5

```
I[3,:] = [2,1,0,1.5]
Top-2 位置 S_3 = {0,3}
```

注意位置1也有一定相关性，但预算k=2，只保留排名前两项。主 Attention 尚未计算。

8.2 主 Attention 自己的 Q/K/V

为便于手算，主 Attention 只有1个头，d_qk=d_v=2：

```
Q[3] = [1,0]
K = [[1,0], [0,1], [-1,0], [0.5,0.5]]   shape [4,2]
V = [[1,0], [0,1], [ 2,2], [1,1]]        shape [4,2]
```

选中 K/V：

```
K_selected = [[1,0], [0.5,0.5]]  shape [2,2]
V_selected = [[1,0], [1,1]]      shape [2,2]
```

主分数为：

```
z[3,0] = dot([1,0],[1,0]) / sqrt(2) = 0.707106781
z[3,3] = dot([1,0],[0.5,0.5]) / sqrt(2) = 0.353553391
```

稳定 Softmax：减去最大值0.707106781，得到[0,−0.353553391]，指数约[1,0.702188501]，和约1.702188501。

```
A_selected = [0.587479001, 0.412520999]   shape [2]
O[3] = 0.587479001×[1,0] + 0.412520999×[1,1]
     = [1,0.412520999]                    shape [2]
```

这里用了两套分数：Indexer 的[2,1.5]只管选人，主分数[0.707107,0.353553]负责如何加权。

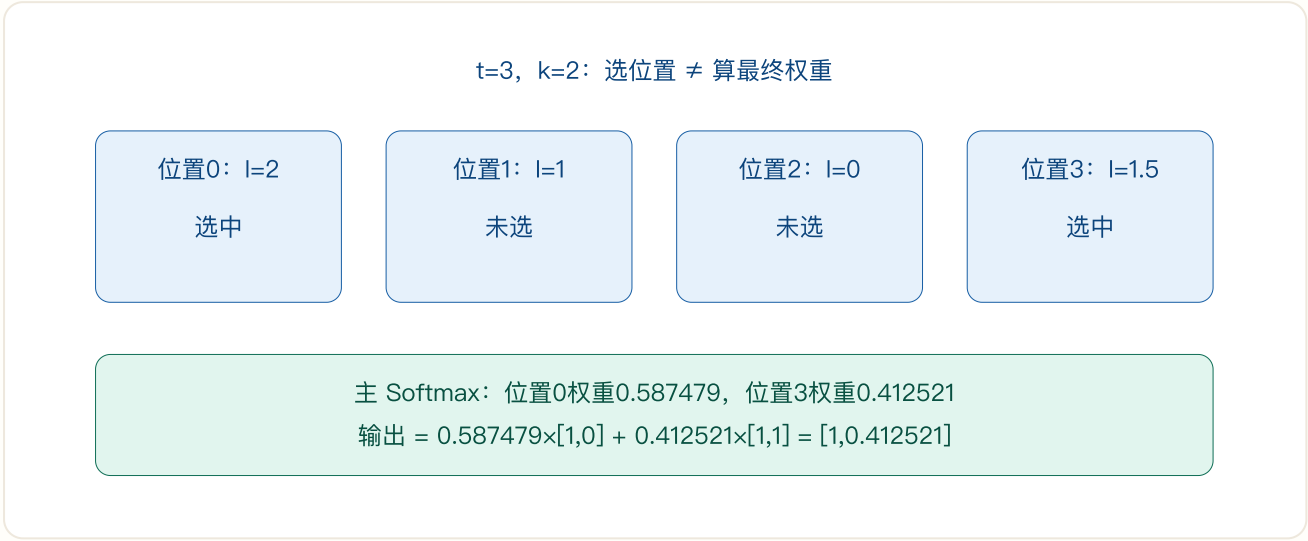
8.3 与 Dense 对比

若主 Attention 访问全部4个位置：

```
z_dense = [0.707107,0,-0.707107,0.353553]
A_dense ≈ [0.410109372,0.202212091,0.099704451,0.287974085]
```

```
O_dense ≈ [0.897492360,0.689595079]
```

稀疏输出[1,0.412521]不同于稠密输出。选择器留下的 Dense 注意力质量约为 0.410109372+0.287974085=0.698083457；归一化这两项就得到本例的稀疏权重。**选得准是质量目标，不是恒等变换。**



9. 训练的难点：Top-k 不可导怎么办

Top-k 返回整数位置。对固定排序的一小段连续参数变化，索引不变；越过排序边界时索引跳变。它不提供让 LM Loss 平滑告诉 Indexer“换成哪个 Token”的普通梯度。

不要把 Top-k 的浮点 values 与 indices 混为一谈：values 可以对选中的元素传播梯度，但用 indices 去 Gather 主 KV，不会自动把语言建模梯度传回产生这些索引的 Indexer 分数。

原始 DSA 不依赖“穿过整数索引反传”，而是采用独立蒸馏目标。

9.1 Dense warm-up：先训练选择器，再真正删边

从已训练好的 Dense 模型开始：主模型冻结、Attention 保持 Dense，只训练新 Indexer。

以主 Attention 的非负归一化权重作为教师信号，跨主头相加后沿可见位置 L1 归一化：

```
p[t,s] = Σ_h A_main[t,h,s] / Σ_r Σ_h A_main[t,h,r]
rho[t,:] = softmax(I[t,:])
L_index = Σ_t KL(stop_gradient(p[t,:]) || rho[t,:])
KL(p||rho) = Σ_s p_s × (log p_s − log rho_s)
```

没有 dropout 干扰且各头行和均为1时，p就是各头 Attention Probability 的平均；不是把可正可负的原始 QK Logits 当成概率相加。

目的：让 Indexer 在不破坏主模型的条件下学会模仿“主 Attention 平时看哪里”。如果直接让随机 Indexer 挑k个位置，主模型可能突然看不到关键上下文。

9.2 Sparse training：打开选择，继续训练主模型

启用 S_t 后，主 Attention 只访问被选集合。Index Loss 也在被选集合上计算：

```
p_selected = 主稀疏 Attention 在 S_t 上跨头聚合后的归一化分布
rho_selected = softmax(I[t,S_t])
L_index = Σ_t KL(stop_gradient(p_selected) || rho_selected)
```

这里教师来自当前主稀疏 Attention，不需要每步额外算一次完整 Dense Teacher。若用截取 Dense 分布做其他实验，必须重新归一化后才是合法概率分布，不能把和小于1的截断向量直接冒充此目标。

梯度路径严格分离：

```
Indexer 输入使用 detach：L_index 不回传到主干 Hidden / 主 Query latent
教师分布使用 stop_gradient：L_index 不通过教师回传主 Attention
L_LM 训练主干及主 Attention
L_index 训练 Indexer 参数
```

“所有参数都训练”表示两个子系统都在更新，不表示两个 Loss 任意流向所有参数，也不表示使用 Straight-Through 或可导 Top-k。

9.3 原始报告的具体训练配方

阶段	主 Attention	更新哪些参数	原始报告配置
Dense warm-up	Dense	仅 Indexer	1000 steps，16×128K tokens/step，LR 1e−3，约2.1B tokens
Sparse continued training	Top-2048	主模型与Indexer，各自Loss	15000 steps，480×128K tokens/step，LR 7.3e−6，约943.7B tokens

这些数字说明稀疏化需要训练投入，不是插入一个Mask就结束；不要把这张表照抄为你当前模型的默认超参数。

10. 用小例子算 Indexer 的学习信号

继续使用 S_3={0,3}、I_selected=[2,1.5]。

```
rho_selected = softmax([2,1.5]) = [0.622459331,0.377540669]
p_selected   = [0.587479001,0.412520999]
L_index      = KL(p_selected || rho_selected) ≈ 0.002574509
```

对 Indexer logits 的局部梯度：

```
dL_index/dI_selected = rho_selected − p_selected
                     = [+0.034980330,−0.034980330]
```

梯度下降会降低位置0索引分数、提高位置3索引分数，让两者比例接近教师。

进一步，令 r_j=dot(qI_j,kI_s)，则局部反向是：

```
dwI_j += dI_s × ReLU(r_j)
dqI_j += dI_s × wI_j × 1[r_j>0] × kI_s
dkI_s += Σ_j dI_s × wI_j × 1[r_j>0] × qI_j
```

ReLU 在0点按实现选择次梯度，本课取0。仅KL选中集合时，未选分数没有这一次直接KL梯度；共享投影参数仍因其他 Query/位置的梯度而变化。由此也能理解 Dense warm-up 的重要性和后续可能出现的选择盲区。

11. 主稀疏 Attention 怎样反向

把本次 Top-k 索引视为固定结构，对选中边执行正常 Attention Backward。

```
dV_s += A_s × dO
dA_s = dot(dO, V_s)
dz_s = A_s × (dA_s − Σ_r A_r dA_r)
dQ += Σ_s dz_s K_s / sqrt(d_qk)
dK_s += dz_s Q / sqrt(d_qk)
```

每个 Query 的局部未选边梯度为0，但未选 Token 并不是整个模型梯度为0：它可能被其他 Query 选中，也可能经残差、MLP 等路径贡献梯度。

数值检验：为本例设一个局部测试目标 L_probe=O[0]+2O[1]，所以dO=[1,2]。这是用于检查链式法则的标量，不是实际语言建模 Loss。

```
a = 0.587479001, b = 0.412520999
dA_selected = [1,3]
dz_selected = [−0.484694849,+0.484694849]
dV0 = [0.587479001,1.174958002]
dV3 = [0.412520999,0.825041998]
dQ  = [−0.171365507,+0.171365507]
dK0 = [−0.342731014,0]
dK3 = [+0.342731014,0]
```

多个 Query 可能同时选择同一个 Key，dK/dV 必须累加：生产 Kernel 的 Scatter、Atomic、归约顺序会影响速度、确定性和低精度误差。固定索引的有限差分可以检查主反向，但不能据此证明索引选择本身可导。

12. FLOPs：不能把 Indexer 从账本里删掉

以下是普通多头逻辑 Attention 的主点积/加权求和模型，FMA计2 FLOPs。忽略投影、RoPE、归约、Top-k、量化开销；生产MLA要按实际kernel维度重新核算。

```
E_dense = S(S+1)/2
E_sparse = Σ_{t=0..S-1} min(k,t+1)
         = k(k+1)/2 + (S-k)k, 假设 S≥k

F_main_dense ≈ 2 B H_a (d_qk+d_v) E_dense
F_main_sparse ≈ 2 B H_a (d_qk+d_v) E_sparse
F_index_dot ≈ 2 B H_i d_i E_dense
```

因此总的核心前向预算：

```
F_DSA ≈ F_main_sparse + F_index_dot + 投影/Top-k/归约/搬运
```

当S不断增大而k固定，主注意力项近线性增长，Indexer点积仍二次增长。实现也可能计算矩形tile再mask，实际执行FLOPs高于有效因果边口径。

对 S=131072、k=2048：

```
最后一个 Query: 131072/2048 = 64 倍交互缩减
整个因果序列:
E_dense = 8,590,000,128
E_sparse = 266,339,328
E_dense/E_sparse ≈ 32.2521
```

所以"64倍"只对应最后一行或非因果满矩形的简化比值，不是整段因果训练的主交互比，更不是端到端加速比。Indexer、投影、MLP/MoE、通信、选择和低效访存都需要时间。短上下文S≤k时主注意力没有删边，还多了Indexer开销。

训练还增加主稀疏反向和Indexer反向；Warm-up仍跑Dense。不能直接拿推理成本曲线来宣称训练Step加速相同比例。

13. 显存：稀疏也不能随意物化中间张量

关键张量口径：

数据	逻辑 Shape	注意事项
Indexer Query	[B,S,H_i,d_i]	可分块；训练保存量看实现
Indexer Key	[B,S,d_i]	额外表示/推理缓存，FP8还含scale
全索引分数	[B,S,S]	只是逻辑shape，不应默认整块常驻

数据	逻辑 Shape	注意事项
Top-k indices	[B,S,k]	int32/int64差2倍；可用变长格式
主概率	[B,H_a,S,k]	不应默认完整物化，可Flash式分块
主 KV	MLA latent 或实际 K/V	未选不等于可全局删除

B=1、S=131072时：完整FP32索引矩阵为$S^2\times4$=64GiB；按矩形[B,S,k]保存int32索引为1GiB，int64为2GiB。主稀疏概率若有128头且用BF16完整保存，也达到64GiB！

因此性能内核需要分块生成分数、选择、Softmax和输出，控制临时张量的生命周期。仅把S^2改成Sk并不能保证峰值小。对比FlashAttention时也不能把本来已经没有S^2激活的Dense基线，误算为节省了一整个Dense概率矩阵。

14. 为什么省 FLOPs 不一定提速

- **Indexer与Top-k**：比较选择未必能像大GEMM一样高效使用Tensor Core。
- **非连续KV访问**：不同Query选择分散位置，Gather导致HBM/L2复用变差。
- **共享与Tile大小**：多个Query heads复用同一KV有助于摊薄加载成本，但跨Query未必选中相同位置。
- **反向热点**：热门Key被大量Query选中，dK/dV归约产生冲突与长尾。
- **Context Parallel通信**：Key在别的Rank上时还要取分数/候选/数据；若沿用全量KV环传，算少了不代表通信少了。
- **示例代码不等于高性能内核**：官方可读推理示例存在先计算Dense主scores再加选择Mask的路径，适合说明语义，不应当作O(Sk)训练kernel证据。

对CP的一种可能设计是各Rank先在本地候选里选top-k，再合并得到全局top-k，随后访问对应KV；前提是跨Rank索引分数完整可比。若分数的索引头也被切分，必须先正确聚合头贡献。原始算法公式本身不替你完成这些分布式设计。

15. 和常见方法的异同

方法	如何决定访问位置	是否改变Dense Attention数学结果	主要特征
FlashAttention	所有mask允许位置	浮点容差内保持	IO优化，不负责内容选择
Sliding Window	固定邻近窗口	是	连续访存，可能漏远程信息
固定Block Sparse	预设或分块布局	通常是	硬件友好但选择粒度粗
DSA	学习Indexer、逐Query Top-k	是	内容感知、token级、需要训练

方法	如何决定访问位置	是否改变Dense Attention数学结果	主要特征
NSA	压缩、选择、滑窗分支组合	是	不是DSA的别名，结构与训练方式不同
MLA	压缩KV表示及投影重参数化	与所用模型定义有关	不等于稀疏，可作为DSA底座

MoE路由选择的是“哪个Expert参数”；DSA选择的是“哪些历史Token”。二者都存在离散选择和负载问题，但DSA的KL教师是主Attention分布，不是MoE的负载均衡目标。

16. 最小语义代码：观察选择、输出与两条Loss

下面仅处理一个Query、一个主头，与前面的数值一致。需要PyTorch；不包含MLA投影、RoPE、FP8、多Rank或高性能稀疏Kernel，不能拿它benchmark DSA性能。

```
import torch

dtype = torch.float64
qI = torch.tensor([[1., 0.], [0., 1.]], dtype=dtype, requires_grad=True)
kI = torch.tensor([[2., 0.], [0., 2.], [-1., -1.], [1., 1.]],
                  dtype=dtype, requires_grad=True)
wI = torch.tensor([1., 0.5], dtype=dtype, requires_grad=True)
Q = torch.tensor([1., 0.], dtype=dtype, requires_grad=True)
K = torch.tensor([[1., 0.], [0., 1.], [-1., 0.], [0.5, 0.5]],
                 dtype=dtype, requires_grad=True)
V = torch.tensor([[1., 0.], [0., 1.], [2., 2.], [1., 1.]],
                 dtype=dtype, requires_grad=True)

I = (torch.relu(qI @ kI.T) * wI[:, None]).sum(dim=0)  # [4]
ids = I.detach().topk(2).indices                    # [2] -> [0,3]
z = (K[ids] @ Q) / (2.0 ** 0.5)                  # [2]
p_main = z.softmax(dim=0)
out = p_main @ V[ids]                              # [2]

# 稀疏训练阶段，教师仅在选中集合上，且停止教师梯度
teacher = p_main.detach()
log_rho = I[ids].log_softmax(dim=0)
loss_index = (teacher * (teacher.log() - log_rho)).sum()
loss_probe = out[0] + 2 * out[1]  # 仅供测试主分支反向
loss_probe.backward()
assert qI.grad is None  # 没有穿过整数索引的主Loss梯度
loss_index.backward()
print(ids.tolist(), out.detach().tolist())
print("index KL:", loss_index.item())
print("main dQ:", Q.grad.tolist())
```

在真实模型中，还必须在生成qI/kI/wI前对传入Indexer的主模型特征detach；本例三组Indexer张量本来就是独立叶子，因此天然隔离。不能因为这段示例没写x.detach就省掉真实网络的隔离。

17. 结合训练工程，应该怎样验收

正确性

1. 固定选中索引，以Dense+相同Mask作为参考，比较O、dQ、dK、dV；此时是在比较同一稀疏函数的两种实现。
2. 再单独比较Indexer打分、ReLU顺序、RoPE布局、量化scale、Top-k与tie-breaking。
3. 验证L_index不会更新主模型、L_LM不会通过索引更新Indexer。
4. 覆盖t<k、S<k、padding、packed segment、空可见行、极端分数和CP边界。
5. 测试真正稀疏模型质量时，再与Dense完整模型对比，不能要求输出逐元素完全一致。

性能与质量

- 单独计时 Indexer Projection、Indexer Dot、Top-k、Sparse Fwd/Bwd、KL、KV Gather、通信。
- 记录allocated/reserved峰值和每层临时张量，而不是只看缓存总容量。
- 监控被选集合覆盖的Dense教师概率质量、长距离召回、不同层/长度的KL与LM验证Loss。
- 收集每个Key被选择次数的分布，以定位反向归约热点。
- 长序列和短序列分开比较吞吐，保持dtype、batch、长度分布和并行布局一致。

对MagiAttention/Flex类工作尤其要记住：**一个实现能表达DSA的Mask，只说明语义可表达；只有执行内核真正跳过未选QK/V交互，才有稀疏计算收益。** 任意token索引、连续range和block mask的转换可能让稀疏集合膨胀，应测实际执行边数/Tile数，而不是仅看理论非零率。

18. 最后串起来

```
Dense模型 → 保持Dense并冻结主干 → KL训练Lightning Indexer
→ 开启每Query Top-k → 主模型用LM Loss适应稀疏上下文
→ Indexer继续从选中集合的主Attention分布学习
→ 高性能稀疏内核真正跳过未选KV
```

核心结论：DSA是“学习检索 + 稀疏主注意力 + 分离训练信号 + 专门内核”的组合，不是简单Mask技巧；主注意力省了交互，但Indexer、索引存储、KV缓存、反向归约与分布式通信仍需入账。

可选自测

1. 为什么主LM Loss不能仅凭Gather的整数索引训练Indexer?

2. S=131072、k=2048时，为什么最后一个Query是64倍交互缩减，而整段因果序列只有约32.25倍?
3. 同一个Key未被当前Query选中，为什么仍可能产生非零总梯度?

继续学习

1. 前置：MLA的MHA/MQA计算模式与权重吸收——准确核算DSA核心维度与缓存。
2. 同层对比：NSA与DSA——块级选择、压缩分支和token级索引的取舍。
3. 下游应用：DSA稀疏Backward与Context Parallel——把选择语义变成真正省算、可扩展的训练实现。

资料与核查边界

- DeepSeek-V3.2-Exp原始技术报告（2025-09-29初版，训练阶段公式3/4）：https://github.com/deepseek-ai/DeepSeek-V3.2-Exp/blob/f4c7a4cd89d0569243b6684ff636c140e0b57608/DeepSeek_V3_2.pdf
- 官方参考实现固定版本：https://github.com/deepseek-ai/DeepSeek-V3.2-Exp/tree/87e509a2e5a100d221c97df52c6e8be7835f0057/inference
- Indexer内核中明确执行dot→ReLU→head权重→跨head归约：https://github.com/deepseek-ai/DeepSeek-V3.2-Exp/blob/87e509a2e5a100d221c97df52c6e8be7835f0057/inference/kernel.py
- 模型配置：https://huggingface.co/deepseek-ai/DeepSeek-V3.2-Exp/blob/main/inference/config_671B_v3.2.json

核查日期：2026-11-03。本文不依赖晚于此日期的V3.2后续报告，也不把可读推理示例当作生产训练内核。Top-k集合的梯度边界、Mask和量化布局必须以目标训练栈为准；性能模型是解析口径，不是GPU实测。